The impact of agile coordination practices on conditions for successful coordination

Tomas Gustavsson

Karlstad Business School,
Karlstad University,
Universitetsgatan 2, 651 88 Karlstad, Sweden
Email: tomas.gustavsson@kau.se

Abstract: Large-scale agile frameworks suggest several practices for coordination between teams in software development projects. These prescribed coordination practices are often described as how to be performed in detail but without explaining the intended impacts. At the same time, agile values emphasise the need for tailoring practices that are not helpful. This study examines how three agile coordination practices influence accountability, predictability, and common understanding, which are important conditions for coordination. A qualitative research strategy was used to investigate three case organisations that implemented the scaled agile framework (SAFe): one bank, one government agency, and one department within the automotive industry. The empirical data consists of 28 semi-structured interviews and observations. The findings show the differing impacts of three investigated agile coordination practices and how the tailoring of practices affects coordination.

Keywords: agile software development; agile practices; agile systems; coordination practices; inter-team coordination; project management.

Reference to this paper should be made as follows: Gustavsson, T. (2023) 'The impact of agile coordination practices on conditions for successful coordination', *Int. J. Agile Systems and Management*, Vol. 16, No. 3, pp.347–367.

Biographical notes: Tomas Gustavsson is an Assistant Professor at Karlstad University. He has a PhD in Information Systems from Karlstad University, an MBA from Stockholm University, and has many years of industry experience as an IT consultant. He is also an author of several prize-winning textbooks on agile development methods. His main research interests are agile ways of working, teamwork, coordination, and leadership in large-scale agile software development.

1 Introduction

Efficient coordination is considered a success factor for software development (Benestad and Hannay, 2011; Dingsøyr et al., 2018; Xu, 2009). However, coordination is one of the biggest challenges associated with large-scale software development today (Bick et al., 2017; Šāblis et al., 2020).

Copyright © 2023 Inderscience Enterprises Ltd.

Practices for coordination have been an important research topic in organisational and management studies for a long time (March and Simon, 1958) and studies on coordination have been undertaken within software development for at least 30 years (Taxén and Riedl, 2016). However, critique on much of the earlier work is that there has been too much focus on the static view of coordination (Jarzabkowski et al., 2012; Okhuysen and Bechky, 2009). As a result, solving emerging coordination needs has been mostly overlooked, and practices to manage them are rarely explained. This is unfortunate since the responses to them are important for efficiency in software development (Taxén and Riedl, 2016).

Okhuysen and Bechky (2009) summarised practices used for coordination into five mechanism types and that different practices can be used for solving the same kind of coordination issue. Therefore, instead of mapping which practice to be the best for one specific problem, they suggested that the main objective for deciding on coordination practices is to support the right conditions needed for successful coordination. Okhuysen and Bechky (2009) call this the integrative perspective on coordination, where the integrated conditions for coordination are accountability, predictability, and common understanding.

Agile methods have become the norm in software development today (Licorish et al., 2016; VersionOne, 2021). However, early agile frameworks were designed without much consideration for scaling problems and focused on solving coordination problems within a single team. Because of the lack of support for managing coordination in larger settings in the first agile frameworks, several frameworks designed for large-scale software development have emerged. The most commonly adopted is the scaled agile framework (SAFe) (VersionOne, 2021). Implementing SAFe often shows positive effects, such as cost savings (Grimaldi et al., 2016), but has also received criticism for being too rigid and focused on too much planning in advance. Static coordination practices are prescribed in detail without explanations of intended impacts (Alqudah and Razali, 2016; Stojanov et al., 2015). At the same time, agile methods emphasise the need for tailoring, adding, or ending practices that are not helpful. Therefore, one way to decide on implementing coordination practices is to understand how the practices support the right conditions needed for successful coordination. Therefore, this paper examines how three large-scale agile coordination practices from the SAFe framework influence accountability, predictability, and common understanding by using the integrative perspective on coordination developed by Okhuysen and Bechky (2009) as the analytic lens. scrum of scrums (SoS), PI planning, and usage of the Program Board are the three practices intended for inter-team coordination investigated in this study.

The paper presents findings from three case studies from organisations who implemented SAFe. During almost two years, 28 interviews were conducted and 379 h of observations were performed for triangulation purposes. Data from observations consisted of notes taken from participation in planning events and meetings.

This paper is structured as follows: Section 2 presents research on coordination, and Section 3 presents agile coordination practices. Section 4 describes the research method, and Section 5 presents the results. Section 6 discusses the results, and the final section presents conclusions, limitations, and future work.

2 Coordination

An early definition of coordination is "integrating or linking together different parts of an organisation to accomplish a collective set of tasks" (Van de Ven et al., 1976, p.322). This definition is relatively broad since it does not specify what 'parts' really mean. Malone and Crowston (1994) suggested that 'parts' rather should be seen as activities that are constrained by dependencies. Therefore, their widely used definition of coordination is 'managing dependencies between activities' (Malone and Crowston, 1994, p.90). One could discuss whether coordination only entails coordination of activities or whether this definition is too narrow. For example, as Dingsøyr et al. (2018) explain, coordination of knowledge is also important. By interpreting 'parts' as both dependent activities and knowledge, the definition by Van de Ven et al. (1976) becomes apparent and useful for this study.

March and Simon (1958) divided the solutions into three areas (or levels): Coordination by standardisation, planning, and feedback or *mutual adjustment* as it was rephrased by Thompson (1967).

Neither March and Simon (1958), Van de Ven et al. (1976), nor Thompson (1967) describe any practices for how to accomplish the vital area of mutual adjustments, only that it is critical for horizontal coordination. The criticism against this earlier work (e.g., Jarzabkowski et al., 2012; Okhuysen and Bechky, 2009) is that the focus is on the static view of coordination where detailed solutions to emerging coordination needs are mostly overlooked.

2.1 Coordination mechanisms and practices

Okhuysen and Bechky (2009) created a framework that summarises five different types of coordination mechanisms and their practices. The first mechanism, Plans and rules, are essential to organising and coordination (March and Simon, 1958). One type of practice described is called *defining responsibility for tasks* which implies explaining the actions that different parties have to take to fulfil a specific common goal. Another type of practice is called *resource allocation*, which often involves using schedules, placing people and activities on a temporal map. The third type of practice is called *developing agreement*, where the final product is a working agreement of how the mission or project will unfold. It serves as an orienting device for the people involved.

The second mechanism, Objects and representations, coordinate by providing information and offer a common referent around which people interact, align their work, and create shared meaning. One type of practice is called *scaffolding* which is the practice of visualising the structure and progress of activities as a reminder of what work still needs to be done: who needs information to complete their work, and who is supposed to deliver that information. *Acknowledging and aligning work* is another type of practice that typically is accomplished by visualising boundary objects which allow different groups to work together without consensus (Leigh Star, 2010). Finally, another type of practice is *direct information sharing*, which often involves using data spreadsheets, large-size computer screens, or physical billboards to share information necessary for the work to progress across different teams.

Roles, the third mechanism, represent expectations associated with social positions and can facilitate behaviour continuity over time. Understanding the relationship between roles and the role structure helps people acquire a general sense of who does what in the

work process (Okhuysen and Bechky, 2009). One type of practice, *monitoring and updating,* is one of the traditional functions of a formal hierarchy. Another practice is called *substitution*, meaning that when individuals understand the tasks linked to each other's roles, they can substitute for one another in task execution. This practice could also be called boundary spanning (Gittell, 2004). For example, a person is appointed to transfer information between different competencies, translating information to create a common perspective to help teams coordinate.

The fourth mechanism, Routines, has historically been an important part of conceptions of organisations and coordination, just like plans and rules (March and Simon, 1958). One practice, called *hand-off work,* means a routine to move activities between people or teams when one group has completed their work. Another practice type is called *task completion*, where a sequence of activities is established, and routines enact a way for interdependent parties to observe progress on the task. A third practice type is called *bringing groups together*, a routine where interdependent parties jointly work on an activity.

The fifth and final mechanism is Proximity which means the physical distance between people. One practice type within this mechanism is to use close proximity to enhance *visibility.* Physical co-location is the primary means of creating visibility in an organisation, created through monitoring practices. In other words, visibility makes monitoring and updating possible. Proximity also leads to *familiarity*, which can enhance coordination (Okhuysen and Bechky, 2009). Increased familiarity leads to stronger relationships and common understanding, which encourages individuals to embrace their interdependence and coordinate more effectively (Gittell, 2002).

2.2 Conditions for coordination

Okhuysen and Bechky (2009) suggest that the main objective for deciding on coordination practices is to support the right conditions needed for successful coordination. The three conditions, accountability, predictability, and common understanding, are the means by which people collectively accomplish their interdependent tasks in the workplace and resolve some of the uncertainties created by interdependence (Okhuysen and Bechky, 2009). Central to their framework is the idea that each condition can be accomplished through a variety of practices based on different coordination mechanisms (see Table 1).

Table 1 Relations between conditions and coordination mechanisms

	Accountability	*Predictability*	*Common understanding*
Plans and rules	Defining responsibilities for tasks	Defining responsibilities for tasks, resource allocation	Developing agreement
Objects and representations	Scaffolding, acknowledging, and aligning work	Scaffolding	Direct information sharing
Roles	Monitoring and updating		Substitution
Routines	Hand-off work	Task completion	Bringing groups together
Proximity	Visibility, familiarity	Familiarity	Familiarity

Source: Based on Okhuysen and Bechky (2009)

Accountability addresses the question: 'who is responsible?' to make clear where the responsibilities lie. By making responsibilities visible, team members become accountable for their contribution while also making other employees accountable for theirs (Okhuysen and Bechky, 2009).

Predictability enables interdependent employees to anticipate subsequent tasks related to an activity by knowing what the elements of the task are and when they should be performed. Predictability allows people to fit their tasks into the whole by anticipating when others will do their work (Okhuysen and Bechky, 2009).

Common understanding supports coordination by providing a shared perspective on the whole and how team members' work fits within the whole. The common understanding is high when team members share knowledge of the tasks that are to be done, how they should take place, and the goals and objectives of the tasks (Okhuysen and Bechky, 2009).

3 Agile coordination practices

Agile software development is often defined by the values and principles described in the Manifesto for Agile Software Development (Beck et al., 2001), which mainly describe values and principles for teamwork for a single team. This has sometimes led to the false assumptions that agile ways of working are suitable only for smaller projects and organisations (Rising and Janoff, 2000). Authors of the first popular agile frameworks, XP (Beck, 2000) and Scrum (Schwaber and Beedle, 2001), only touched on the challenges of scaling the frameworks.

In large projects, coordination across teams is a challenge (Bick et al., 2017). When agile projects involve several teams, teams must coordinate via spanning activities, spanning objects, and spanning roles (Strode et al., 2012). As larger organisations started implementing agile methods, agile practices containing spanning activities and spanning objects were developed. Also, several spanning roles were introduced, such as Area Product Owners and Release Train Engineers (Leffingwell, 2007). These practices are often suggested as parts of agile frameworks intended for agile work in large-scale settings, such as SAFe (Leffingwell, 2007), LeSS (Larman and Vodde, 2010), DAD (Ambler and Lines, 2012), and the Spotify Model (Salameh and Bass, 2020). The most commonly adopted of these frameworks is SAFe (VersionOne, 2021) which is described in the following subsection.

3.1 The scaled agile framework (SAFe)

SAFe is considered a rather complex framework, and it incorporates a vast set of roles, practices and process elements (Kowalzyk et al., 2022). SAFe is scalable and was primarily developed for organising and managing agile practices in large enterprises (Razzak et al. (2018). Reviews show that among studies on large-scale agile, SAFe has received the most research attention compared with other large-scale frameworks (Uludag et al., 2022).

The framework describes agile ways of working on different levels, starting from the team level to the program level, portfolio level, and organisational level (Kowalzyk et al., 2022). All teams are part of an agile release train (ART) which is the central construct of the program level. All teams on an ART are synchronised by working in joint Product

Increments (PI) and delivers a continuous flow of incremental releases of value (Razzak et al. (2018). The PI for the ART corresponds to sprints at the team level, but with a longer timeframe (Kowalzyk et al., 2022), often a time period of 4–6 sprints (Leffingwell, 2007). In order to synchronise the teams in an ART, coordination is needed to identify and manage dependencies between teams and the surrounding organisation (Razzak et al. (2018).

SAFe provides several practices for an ART of which three of them are specifically intended for inter-team coordination within the ART (Scaled Agile, 2021). The PI planning practice is intended to find dependencies between the teams for the upcoming sprints in the PI. The use of a program board is intended to visualise and keep track of these dependencies and the SoS practice is a meeting intended to solve dependency issues during the sprints in the PI. These three practices are described in the following subsections in more detail.

3.2 Scrum of scrums

The SoS practice is explained in the SAFe framework but it was originally proposed by Schwaber (2004) to manage dependency issues between several teams in the Scrum framework. SoS is described as a weekly meeting between all Scrum Masters in a product line to discuss and solve dependency issues between teams (Sutherland, 2001). The suggested agenda contains the following questions to be answered by each person in the meeting:

1 What did my team do that is relevant to other teams?

2 What will my team do that is relevant?

3 What are my team's obstacles that other teams should know about or can help with? (Larman and Vodde, 2010).

Adopting the practice in this form has been criticised (Paasivaara et al., 2012). Allowing only Scrum Masters to these meetings puts a risk of talking about solutions rather than solving them. Practitioners have since stressed that the meetings should be a place for solving coordination issues, not a meeting for managers (Larman and Vodde, 2010), and put forth the need for allowing any team member with a coordination issue to participate in SoS meetings (Paasivaara et al., 2012). Another criticism of the practice has been that a weekly meeting does not solve problems between teams fast enough. Since the Scrum teams meet daily to sort out problems, called the Daily Scrum (Schwaber, 2004), waiting for a whole week to sort out problems between teams is much too slow. Therefore, SoS is a coordination practice intended to manage emerging coordination needs.

3.3 PI planning

Misalignment of planned work between teams is a common problem in large-scale agile software development (Bick et al., 2017). The degree of detail often differs between detailed work in the single team plans, while more abstract, coarse-grained work is presented on an inter-team level (Bick et al., 2017).

PI planning is a practice intended for several teams, and stakeholders, working towards a common goal to make a shared plan (Scaled Agile, 2021). The practice intends to align all the teams' work for a specific period into the future, often 10–15 weeks

(Scaled Agile, 2021). The planning should follow a strict cadence, i.e., if a 12-week plan is chosen, all successive plans should be made for 12 weeks in the future. The plan should contain several iterations or sprints, ending with a final innovation and planning (IP) sprint where the teams reflect, experiment, and perform the next PI planning.

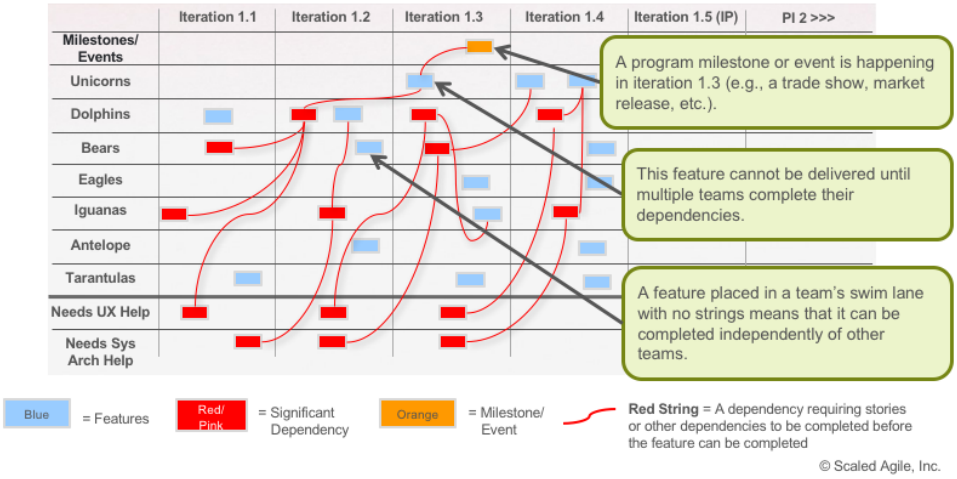
The planning event often lasts for two days and contains overall presentations and team breakouts, where each team plans their work individually. SAFe (Scaled Agile, 2021) suggests a meeting agenda where the first half of the day contains presentations from stakeholders and senior managers. The rest of the day and the first hours of day two contain team breakouts where each team plans for themselves. SAFe suggests allocating roughly 30% of team breakout time during the two days (Scaled Agile, 2021). The remaining part of the second day contains presentations of plans, risk management, and possibly re-planning.

The event's outcome is a roadmap showing the expected work estimated for all teams (Dalton, 2019). Besides the roadmap, another goal of PI planning is to create a visualisation of dependencies between teams, also known as the program board (Scaled Agile, 2021). The PI planning event is a coordination practice intended to manage planned, static coordination needs.

3.4 The program board practice

A program board, sometimes called a planning board (Bajpai et al., 2020), shows dependencies between activities on a physical or digital board (Scaled Agile, 2021). The dependencies on the program board are visualised by displaying the different teams involved in the work on the Y-axis and the work for the selected time span along the X-axis. Dependencies between activities could be shown by using red strings, as shown in Figure 1. The program board should contain milestones, work on features, and significant dependencies between features, according to SAFe (Scaled Agile, 2021). There is no definition in SAFe of what 'significant' entails other than that a feature cannot be delivered until dependent features are completed.

Figure 1 Example of a program board (see online version for colours)



Source: ScaledAgile (2021)

By implementing routines for managing the program board, the current situation of dependencies can be visualised and kept track of (Scaled Agile, 2021). The program board should be complete at the end of the PI planning event and show all known significant dependencies for the upcoming planning period. Then, as work begins, the program board may, or may not, be updated. This depends upon the tooling in place and the needs of the teams (ScaledAgile, 2021). If it is not updated, it is only used for planned coordination, but if the program board is updated, using the program board is also a coordination practice intended to manage emerging coordination needs.

4 Method

A qualitative approach was chosen, which is considered suitable for developing a deep understanding of the coordination practices performed within their natural organisational setting (Yin, 2017). The intention is to understand the views from the perspective of team members and surrounding roles on how the three selected agile practices had an impact on conditions for coordination. The teams had implemented various agile practices, but the focus of this study relates solely to the three selected agile coordination practices presented in the previous section. They were chosen since they are explicitly developed to manage inter-team coordination. Three exploratory case studies were conducted to identify differences and similarities across teams in different settings.

4.1 Case descriptions

The first case, called GovAgency, is a project started in a software development department within a large Government Agency in Sweden (more than 13,000 employees). The project was started as a pilot project for implementing SAFe with the ambition to get best-practice knowledge for further implementations in the rest of the organisation. The software developed at GovAgency is web-based services for Swedish citizens. GovAgency started using Scrum and agile ways of working in 2013 and consisted of 8 teams with 70 employees developing software to be used both in-house, for administrators, and citizens in need of their services. GovAgency started implementing SAFe at the end of March 2017.

The second case, called BizBank, is a software development department in one of the largest business banks in Sweden (more than 15,000 employees). BizBank develops an in-house software service to manage client fund portfolios for administrators. BizBank reorganised itself by expanding its teams from purely development teams into teams responsible for both development and operations. The studied department in BizBank introduced agile ways of working in dedicated self-organising development teams during 2014. BizBank then started implementing SAFe in April 2017 when they also reorganized the teams. BizBank consisted of 7 DevOps teams, developers and operations working together (Teixeira et al., 2020), with 42 employees in the department.

The third case, called MotorDev, is a product development department within the Automotive industry. The department consisted of teams residing in four different countries, where most of the teams were located in Sweden. MotorDev started using Scrum and agile ways of working in 2012 and started implementing SAFe in January 2017. MotorDev develops an in-house software platform and services in a motor vehicle for a product-service-system (PSS) with an agile way of working to achieve customer-

centricity (Einsiedler et al., 2022). The department consisted of 24 development teams divided into three areas, with most software developers but also some hardware developers. Around 80% of the product development work is software development and 20% hardware development. Some of the teams were rather small, so the total amount of employees was 141 people.

Before, all three organisations worked according to Scrum, in self-organised teams without formal practices to coordinate between teams. Instead, all dependency issues were dealt with in an ad-hoc manner by what Thompson (1967) calls "mutual adjustments".

4.2 Data collection

Semi-structured interview protocols were used in interviews conducted with informants from various hierarchical levels such as team members (N = 14), Scrum Masters (N = 4), Product Owners (N = 3), Release Train Engineers (RTE) (N = 4), Agile coaches (N = 2), and one Product Manager (N = 1).

Observations, using field notes and photographs, interview protocols, and transcripts, were used for triangulation purposes. In addition, reading archival records, mostly memoranda from meetings, also helped capture details of the performed practices. Observations and interviews in the various cases are presented in Table 2. Data was collected from the starting point of implementing the large-scale agile coordination practices in the organisations (from January to April 2017) until November 2018, a period of almost two years.

Table 2 Observations and interviews

Case organisation	*Hours of observation*	*Number of interviews*
GovAgency	113	6
BizBank	70	8
MotorDev	196	14
Total	379	28

4.3 Data analysis

Although several practices contribute to coordination between teams (Dietrich et al., 2013; Dingsøyr et al., 2018) in large-scale agile software development environments, three agile coordination practices were analysed in this study: SoS, PI Planning, and the use of the program board. These three agile practices were chosen since they are intended explicitly for coordination between teams in an ART (Scaled Agile, 2021).

A deductive coding approach was performed in two iterations. First, data were analysed by coding interview transcripts and field notes based on the type of agile coordination practice used and whether it was used for planned or/and emerging coordination needs. In the second round, codes were organised in categories based on the impact and, specifically, whether there was an impact on accountability, predictability, or common understanding. An example of coded quotes belonging to one category is presented in Table 3.

Table 3 Coding example

Category	*Quotes*
PI Planning, Impacts on Predictability, MotorDev	"it is that it has become more clear… it has become easier for us to actually communicate back to stakeholders what we can and cannot do" (RTE1)
	"PI planning is very structured and I like that… it helps us to plan and see what's coming up in advance. You could say it's more predictable now than it's been before" (Dev3)
	"You feel more comfortable when you go back [to the stakeholder] and communicate that 'we will not be able to do these things because we are good at estimating what we can fit in a PI'" (PO2)

The amount, or level, of impact on the conditions is based on expressed perceived impacts by the interviewees. One exception was the impact on predictability, where the amount of completed work was also used to measure impact. Also, the coded interview transcripts and field notes were compared with documents such as meeting minutes and status reports. Differences or uncertainties in interpretations lead to further investigation through complementary discussions and informal interviews during on-site visits.

5 Findings and analysis

In this chapter, findings from the cases studied are presented and analysed in three subchapters. Each subchapter focus on each of the chosen agile coordination practices. For each coordination practice, findings from each case organisation are analysed.

5.1 The scrum of scrums practice

At GovAgency, SoS meetings were held once a week on a strict cadence, always on Wednesdays from 10.30 to 11.00. Several managers and relevant stakeholders attended the meetings, apart from the chairman (an RTE) and Scrum Masters. However, by meeting as seldom as once a week, participants expressed only marginal improvements of predictability: "It would have been much better if we saw them [meeting attendees in SoS] more often. Then we wouldn't have to wait for decisions and the plans wouldn't fail as often." (SM1, GovAgency).

The meeting was held virtually, and anyone at GovAgency could listen in during the meetings. The chairman explained what the meetings looked like:

> "So, the agenda is [the] same as always. First, we have information from [managers], our architect, product management, our test manager, and the [Business Development] team. Then every team presents information that could be important for other teams and dependencies. Then, we address risks." (RTE, GovAgency)

The intended aim of SoS (Sutherland, 2001; ScaledAgile, 2021) is to be a meeting to help out between teams, not an information-sharing meeting. But at GovAgency, SoS meetings were primarily seen as an arena for information sharing, and secondly as a meeting for highlighting risks and dependencies. This meant that much information was shared but not as many upcoming risks were addressed, as this quote from a Scrum

Master shows: "It's good to hear more about how things are going for the teams that are not in [City X]. Information from the management is also good, but otherwise I do not think we talk enough about risks we need to solve." (SM2, GovAgency).

At GovAgency, managers and stakeholders participated to be able to inform the meeting participants about important decisions and status on areas surrounding the project. Due to this information sharing from several roles, the common understanding was highly improved. Also, more decisions could be made since more roles were attended during these meetings besides Scrum Masters or team representatives. As a result, decisions on responsibilities and changes in priorities led to highly improved accountability.

BizBank chose to conduct SoS meetings in the same format as the Daily standup meeting (Schwaber, 2004), with a fixed timebox of 15 minutes per day, always between 12.45 and 13.00. The time was chosen based on that many other kinds of meetings at BizBank started at 13.00. According to three of the respondents, the focus in the meetings was primarily to help out between teams, and every participant mainly talked about what their team needed help with. Based on the obstacles in the single team, additional team members besides the Scrum Master sometimes joined the meeting to further explain the details of the problems. If some issues could not be solved during the meeting, parties involved stayed to resolve issues in a 'meet after,' whenever necessary.

Due to this setup of daily SoS meetings with daily updates of progress, predictability was highly improved. Also, BizBank chose to expand the SoS meeting about once a week to follow up on progress. In this meeting, managers and stakeholders also attended. One respondent explained that at first, they invited managers and stakeholder but did not know if they would turn up in the meetings:

> "I was a little unsure in the beginning like 'will this be something that is well attended?' And from the beginning it was only Scrum Masters and Product Owners, but then it turned out that many were interested in hearing about how it went and risks … And often when a question was raised there was like 'Well, now we would need Charles here who knows this'." (RTE, BizBank)

Every Scrum Master reported and discussed the progress of the teams' work, and the program board was updated in detail. This prolonged meeting format with additional roles attending made it possible to make more decisions on changed responsibilities and priorities, highly improved accountability, and common understanding.

The implementation of SoS meetings at MotorDev was not highly prioritised early in the process. One employee explains: "We started with SoS [meetings] once a week, then every second week and now once a sprint. They got stuck in just talking about what they had done. Status reporting. [We] tried to foster a mindset of focusing on risks in the team instead of babbling too much about what they had done." (RTE at MotorDev)

Apart from how often SoS meetings were conducted, they were always performed in the same way, letting one participant at a time address what they had done since the last meeting and if they had problems or saw possible problems. The participants were always Scrum Masters representing each team. A former project manager was appointed as chairman for all meetings. Managers viewed the meetings as an arena for highlighting risks, while team representatives saw it mainly as a place to report progress. According to four of the respondents, none regarded the meeting as a place specifically to resolve dependency issues. Therefore, the practice only marginally improved accountability and predictability. Instead, with a focus on status reporting, the common understanding was

the condition mostly improved by the practice. A summary of how the conditions needed for coordination was affected is displayed in Table 4.

Table 4 Conditions affected by the scrum of scrums practice

Condition	*GovAgency*	*BizBank*	*MotorDev*
Accountability	Highly improved	Highly improved	Marginally improved
Predictability	Marginally improved	Highly improved	Marginally improved
Common understanding	Highly improved	Highly improved	Highly improved

5.2 The PI planning practice

At GovAgency, the length of the planning period was tailored based on the number of available man-hours, according to two of the respondents. During the summer months, when most people were on holiday leave for a couple of weeks, the planning period was lengthened to compensate for less available man-hours in the teams. A similar condition went for Christmas and Easter. Therefore, the planning period covered everything from 12–15 weeks, depending on when planning was performed. In other words, the planning cadence at GovAgency was the number of man-hours to be planned for, not a strict calendar cadence. The department manager at GovAgency did not allow an IP sprint, only a short (less than 2 h) retrospective meeting and two days for PI planning, based on the standard agenda suggested in SAFe (Scaled Agile, 2021). The reason for not allowing a sprint for inspection and adaption, according to two respondents, was that it was seen as a waste of time and money to spend so much time on non-productive activities. This was the view from management, and it was not shared with RTE, Product Owners, and Scrum Masters, who argued that GovAgency would benefit from an IP sprint. One respondent explains: "They say that it's important for us to reflect and learn from each other, but when we ask why we can't do a proper IP-sprint, they say that it won't be funded from top management" (SM1, GovAgency). According to the teams, this lack of time for reflection and innovation in conjunction with the PI planning practice meant that common understanding was only marginally improved. Team members complained that planning felt 'non-agile' since the workshop was conducted in the same way each time. Also, plans were not updated during sprints, making the whole planning period seem like a commitment instead of a plan with the intended flexibility at the start of each new sprint. With such a long planning period and a loss of flexibility, accountability was only marginally improved. Regarding predictability, planning precision was investigated by measuring the amount of planned work being completed. No data was available for the first planning period, but 48.5% of planned work was completed during the second planning period. During the third planned period, only 47.5% was completed, and the percentage for the fifth planned period was 55.7%. With such small changes in planning precision, predictability could not be considered as being improved.

In the first PI planning at BizBank, the department manager only allowed half a day of joint planning of which the agile coach at place responded in anger: "with so little time, we can't plan for more than two sprints. Two sprints! That's so little!" (AC, BizBank). BizBank had decided on two and a half-week sprints, so this would only mean a five-week plan. But after the anger had subsided, the agile coach changed how he thought about the PI planning practice regarding learning. Instead of learning to plan

according to a strict cadence, the organisation will learn faster about how to perform PI planning since they will have the next PI planning as soon as five weeks after the first attempt. Shorter learning loops will cause faster learning, was the new logic as the coach explained it:

> "When we wanted to do the first [PI] planning, the reason to why we only planned for two sprints was actually because we were not allowed to have a longer planning workshop, only four hours. Then I thought, 'well, that's actually great, [it] means [that] we will learn how to plan faster'. That's why we have expanded the [planning period] bit by bit, instead of having a long fixed cadence." (AC, BizBank)

Also, with feedback on the performance of the PI planning practice, tailoring of the practice could be performed more often. With this new view of the short planning period as a way to increase learning, expanding the planning period each time caused the structure of the PI planning agenda to variate a lot from the first up until the fourth PI planning. During the first two times, only two sprints were planned. The third time, three sprints were planned for, and the fourth time, BizBank finally decided on a fixed cadence of four sprints plan for the future. This successive expansion of planning length meant highly improved accountability already from the first PI plan. According to four of the respondents, it was easy to understand who was responsible for what in every detail with a short planning period. This also highly improved common understanding. With the short feedback loops, predictability was also highly improved. On the first studied planning event, 60% of the planned work was completed. For the fifth PI plan, covering four sprints, 87% of all the work planned for the period was completed.

MotorDev decided from the very beginning to work in a fixed cadence of ten-week planning periods. The reason for having a strict cadence was explained in the same way by three of the respondents. By having a strict rhythm, team members will be better at predicting and estimating, and that will help MotorDev to be more precise in predictions towards stakeholders. Improving predictability was, as seen, an intended benefit for using the PI planning practice on a set cadence.

The managers at MotorDev thought that one and a half days would probably be enough time for the teams to finalise their plans and decided to conduct the workshop using a little less time for each item than SAFe prescribes in their suggested planning agenda (ScaledAgile, 2021). A one-week IP sprint was allowed to be able to experiment, reflect and perform PI planning. The allowed time for the IP sprint, therefore, highly improved the common understanding. MotorDev always used one and a half-day for planning but made many changes in the different agenda items to tailor according to feedback from employees. The amount of time spent on team breakout time changed along the way. In the beginning, 45% of the time was spent on team breakout planning, but this changed after the criticism of spending too much time in presentations. During the tenth performed PI planning, 65% of the time was spent on team breakout planning, and the format of the introductory presentations was much shortened to only contain the most relevant information. As the amount of breakout time increased, planning precision increased, which highly improved predictability. From 78.4% of planned work being completed during the third time PI planning was performed, up to the full 100% being completed during the tenth time PI planning was performed. The improved predictability was also mentioned in interviews such as this example shows: "PI planning is very structured and I like that. It helps us to plan and see what's coming up in advance. You could say it's more predictable now than it's been before" (Developer3, MotorDev).

The joint planning of teams and stakeholders highly improved accountability for the upcoming 10 weeks of work. At MotorDev, employees did not perceive the shortened presentations to harm common understanding. Instead, the carefully chosen presentations highly improved common understanding. A summary of how the conditions needed for coordination was affected is displayed in Table 5.

Table 5 Conditions affected by the PI planning practice

Condition	*GovAgency*	*BizBank*	*MotorDev*
Accountability	Marginally improved	Highly improved	Highly improved
Predictability	Not improved	Highly improved	Highly improved
Common understanding	Marginally improved	Highly improved	Highly improved

5.3 The program board practice

During the first PI planning at GovAgency, all milestones, work on software features, and dependencies were presented on the program board. The board finally showed that all of the planned work to build features had dependencies either with other teams or other departments. Every single feature had dependencies outside the team. Managers and stakeholders refused to believe that none of the work could be performed without dependencies outside the team. Therefore they questioned the visualised dependencies and asked the teams to present some of them. Answers showed that the reason for the overwhelming amount of dependencies was due to different understandings of what a dependency really means. Team members presented a dependency on the board when the team only needed information from someone outside the team, i.e., a pooled dependency (Thompson, 1967). On the other hand, managers and stakeholders suggested that a dependency should only be presented if there was a need for a specific systems function to be fulfilled before the dependent function could be developed, i.e., a sequential dependency (Thompson, 1967). After a brief discussion at the end of the PI planning, all parties agreed on a common definition: a dependency should be visualised when a team needs something delivered from another team. In other words, only sequential dependencies were to be visualised, not pooled dependencies. This definition caused almost all visualised dependencies on the program board to be removed. Only eleven dependencies to other departments were left. Suddenly, there were no dependencies between the cooperating teams at GovAgency, only towards teams in other departments. With this definition, dependencies between people based on information needs were no longer visualised. This caused several problems, as this quote from a developer exemplifies:

> "…and I asked him, how should I know that you needed that information? I didn't know that you needed to know about the details in that system. How could I know that? No one told me about it!" (Developer1, GovAgency)

Four of the respondents, two Scrum Masters and two developers, expressed their frustration regarding the decision of not allowing pooled dependencies to be visualised on the board. With the lack of dependencies based on information needs visualised on the program board, it did not improve accountability, predictability, or common understanding. The program board was not updated during the sprints. Since so few dependencies between teams were recognised during PI planning, GovAgency abandoned

using the program board during their fourth PI planning. Instead, teams recorded dependencies in their team plans.

At BizBank, the program board contained all features, dependencies between features, and milestones. Instead of updating the program board during the daily SoS meetings, BizBank updated the board during the first day of every new sprint and during the weekly expanded SoS meetings with managers and stakeholders attending. Each time the program board was updated, colour-coded notes were used to visualise changes. For example, completed work was marked with blue notes and postponed work with orange notes. Sometimes, upcoming work was exchanged between teams, and information about this was added on the orange notes. These visual clarifications highly improved predictability and accountability. Common understanding was not affected.

MotorDev decided from the start only to use the program board to visualise the work with dependencies between teams and other departments, not to show all of the upcoming work. Significant milestones were also displayed. Since all upcoming work was not visualised, the program board did not improve common understanding.

The program board was only updated during PI planning and was not kept up to date between these planning events. This meant that neither accountability nor predictability was improved. A summary of how the conditions needed for coordination was affected is displayed in Table 6.

Table 6 Conditions affected by the program board practice

Condition	*GovAgency*	*BizBank*	*MotorDev*
Accountability	Not improved	Highly improved	Not improved
Predictability	Not improved	Highly improved	Not improved
Common understanding	Not improved	Not improved	Not improved

6 Discussion

In this study, three inter-team coordination practices and their impact on conditions for coordination have been investigated in three case organisations. When investigating the SoS practice, the findings from this study suggests that using the standardised way of performing SoS meetings only on a weekly basis, the benefits of improved accountability and predictability might be lost or only marginally affected. Conducting SoS meetings on a daily basis might instead have a much higher impact on both predictability as well as common understanding. Previous studies on large-scale agile software development show challenges in conducting SoS meetings to solve inter-team coordination issues (Paasivaara et al., 2012; Paasivaara and Lassenius, 2016; Kalenda et al., 2018). The meeting cadence varies between organisations. Some organisations conduct the SoS meetings daily (Paasivaara and Lassenius, 2016), some two to three times per week (Dingsøyr et al., 2018), and some on a weekly basis (Kalenda et al., 2018). The cadence is often tailored over time, where some increase the number of meetings per week and some conduct them more seldom (Kalenda et al., 2018).

The prescribed way of performing SoS meetings is with only Scrum Masters or team representatives attending (Larman and Vodde, 2010). In this study, adding stakeholders such as Product Owners and Managers made it possible to make decisions on changed

responsibilities and priorities. The impact of such an extension of SoS members attending suggests an impact of highly improved accountability. This finding contradicts the proposed problem of the SoS meeting becoming a meeting for managers (Larman and Vodde, 2010). There are, however, problems reported regarding how the SoS meetings are performed, besides the roles of the participants. The meeting needs to have a long enough duration to enable discussions and solve dependencies (Stray et al., 2022). Otherwise, as described in Paasivaara and Lassenius (2016), when the number of teams grows, participants might start just stating "nothing to report", even though that might not be true.

Teams in large-scale software development often lack the high-level picture to resolve dependency issues with other teams (Bick et al., 2017). The PI planning practice is intended to create such an overall view of dependencies and coordination needs. Research on the PI planning practice shows that inter-team dependencies are addressed earlier, and alignment between the teams is improved (Brenner and Wunder, 2015). In this study, PI planning improved accountability, predictability, and common understanding in two of the investigated cases. In the third case, two conditions were only marginally improved, and predictability was not improved. The findings suggest that conducting a PI planning directly after ending the previous planning period, without time for reflections and experimentation, might damage common understanding, which was the case at GovAgency. Also, performing the PI planning in the same manner without tailoring the format based on feedback might have negative consequences. In the BizBank and MotorDev cases, extending the amount of time allowed for team breakout planning highly improved accountability and predictability. This impact was not seen in the GovAgency case, where PI planning always followed the same format. But tailoring the planning format is not suggested in SAFe (ScaledAgile, 2021), where only the standard format is prescribed.

The program board may, or may not, be updated between PI planning events depending on the tooling in place and the needs of the teams, according to SAFe (Scaled Agile, 2021). Two of the investigated cases, GovAgency and MotorDev, did not update the board continuously and experienced no improvements on accountability and predictability. The third case organisation, Bank, updated the board continuously, and accountability and predictability were highly improved. These findings suggest that not updating the program board might impair accountability and predictability. This possible drawback is not presented in SAFe, only that it is a choice (Scaled Agile, 2021).

A study of the use of the program board presented in Bajpai et al. (2020) shows the problem of granularity. SAFe suggests visualising dependencies between features (Scaled Agile, 2021) and not dependencies between user stories. Bajpai et al. (2020) show that such a practice results in a systematic underrepresentation of the need for interactions between teams to solve dependency issues. An issue similar to finding the right level of granularity was found in this study. The finding from investigating the program board use practice in this study was difficulties in defining what types of dependencies to visualise. The problem was only evident at GovAgency, where only sequential dependencies were visualised but not pooled dependencies (Thompson, 1967). Withdrawing pooled dependencies from the program board caused coordination problems at GovAgency. At Bank and MotorDev, however, there were no discussions on types of dependencies to visualise, and coordination problems were few. Whether this was due to a mutual understanding of which dependencies to visualise or whether pooled dependencies were

scarce, and did not cause problems, is unknown. More research on coordination difficulties due to different types of dependencies in software development is needed.

The case organisation that gained the most improvements from using the program board, BizBank, was also the one who gained the most benefits from the SoS practice. Investigating the differences between BizBank and the other cases shows that the SoS practice was used more frequently at BizBank. Fifteen-minute SoS meetings were performed daily with an additional weekly prolonged meeting with managers and stakeholders attending where the program board was updated. This finding suggests a cumulative positive effect of more frequent SoS meetings and keeping an updated program board.

7 Conclusion

Large-scale agile frameworks suggest several practices to coordinate work between teams. These agile practices are often prescriptions in detail without explanations of the intended impacts of the practices. At the same time, agile values emphasise the need for tailoring practices that are not helpful. This study investigates whether three agile coordination practices support the right conditions needed for successful coordination. The study aims at answering how SoS, PI planning, and the use of the program board influence accountability, predictability, and common understanding.

Previous studies shows differing perceived satisfaction of practicing SoS (Paasivaara et al., 2012; Paasivaara and Lassenius, 2016; Kalenda et al., 2018). Performing SoS weekly with Scrum Masters or team representatives attending improves common understanding but has limited impact on accountability and predictability, according to this study. However, with a setup of daily 15-min meetings and a prolonged weekly meeting with managers and stakeholders attending, this study suggests all three conditions might become highly improved.

Research on the PI planning practice suggests that alignment between the teams is improved and inter-team dependencies are addressed earlier (Brenner and Wunder, 2015). However, conducting a PI planning directly after ending the previous planning period, without time for reflections and experimentation, might damage common understanding. Instead, tailoring the meeting agenda based on joint reflections, such as extending the amount of time allowed for team breakout planning, has a positive impact on accountability and predictability. Also, planning based on available man-hours instead of keeping a fixed cadence, such as a 10 weeks plan at a time, might not improve predictability.

The program board is intended to visualise dependencies between teams. As presented in previous research (Bajpai et al., 2020), this study also shows the importance of granularity and types of dependencies to visualise. Also, keeping the program board up to date, instead of only updating it during each PI planning, could have a positive impact on accountability and predictability. This finding is not highlighted in previous research or in SAFe recommendations which only describes that the board may, or may not, be updated. This study also suggest that there is a cumulative positive effect of more frequent SoS meetings and keeping an updated program board.

These investigated impacts of agile coordination practices and examples on tailoring can help organisations in choosing which agile coordination practices to implement as well as how to tailor them.

7.1 Limitations of the study

Inherent to any empirical study, this study has limitations. One such limitation is that the perceived impacts are studied in this research, not the measured impacts. Only planning precision, i.e., features delivered compared to what was planned, is measured to investigate impacts on predictability. This is an important metric, though, as metrics related to the business value delivered and sprint goal success are the most relevant (Almeida and Carneiro, 2021). However, one limitation might therefore be the limited validity of the observed cause-effect relationships between the implemented agile coordination practices and their impacts. It can also not be ruled out that other factors, such as other agile practices or company specific routines and procedures, might have had an influence on the perceived impacts. The case organisations operate in different industries with differing conditions such as strictness of regulations and demands from stakeholders. Even though this study investigate the internal work between teams, it cannot be ruled out that external factors also might have an influence on the perceived impacts of the implemented coordination practices.

Another limitation relates to the reliance on interviews as the major data source. In order to improve validity, respondents were chosen based on having experience with and knowledge of agile software development. Besides a large number of interviews (28), triangulation helped with reducing the study's bias by taking observation notes and collecting meeting memoranda. To ensure reliability, interview transcripts and observation notes were sent to employees so that they could check for errors or omissions.

As data was collected from only three case organisations, the results should be interpreted cautiously and may not be directly generalised to other contexts. Further research with a larger sample of cases is required to validate the results.

7.2 Future work

Since the study investigates perceived improvements on conditions for successful coordination, future studies could focus on measuring these impacts. Improvements in predictability are measured by using data on planning precision in this study, so finding relevant measurements for accountability and common understanding could be of interest to increase our knowledge on the impacts of agile coordination practices.

References

Almeida, F. and Carneiro, P. (2021) 'Performance metrics in scrum software engineering companies', *International Journal of Agile Systems and Management*, Vol. 14, No. 2, pp.205–223.

Alqudah, M. and Razali, R. (2016) 'A review of scaling agile methods in large software development', *International Journal on Advanced Science, Engineering, and Information Technology*, Vol. 6, No. 6, pp.828–837.

Ambler, S.W. and Lines, M. (2012) *Disciplined Agile Delivery: A Practitioner's Guide to Agile Software Delivery in the Enterprise*, IBM Press, Indianapolis.

Bajpai, S., Joglekar, N. and Eppinger, S. (2020) 'Enhancing visibility in agile program increment DSMs', *Proceedings of the 22nd International DSM Conference (DSM 2020)*, Cambridge, Massachusetts, pp.1–10.

Beck, K. (2000) *Extreme Programming Explained*, Addison-Wesley, Indianapolis.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R.C., Mellor, S., Schwaber, K., Sutherland, J. and Thomas, D. (2001) *Manifesto for Agile Software Development*, Available at. http://www.agilemanifesto.org (Accessed 5 October, 2021).

Benestad, H.C. and Hannay, J.E. (2011) 'A comparison of model-based and judgment based release planning in incremental software projects', in Taylor, R.N., Gall, H. and Medvidović, N. (Eds.): *2011 33rd International Conference on Software Engineering, ICSE*, IEEE, Waikiki, HI, USA, pp.766–775.

Bick, S., Spohrer, K., Hoda, R., Scheerer, A. and Heinzl, A. (2017) 'Coordination challenges in large-scale software development: a case study of planning misalignment in hybrid settings', *IEEE Transactions on Software Engineering*, Vol. 44, No. 10, pp.932–950.

Brenner, R. and Wunder, S. (2015) 'Scaled agile framework: presentation and real world example', *2015 IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops* (*ICSTW*), IEEE, Graz, Austria, pp.1–2.

Dalton, J. (2019) *Great Big Agile*, Apress, Berkeley.

Dietrich, P., Kujala, J. and Artto, K. (2013) 'Inter-team coordination patterns and outcomes in multi-team projects', *Project Management Journal*, Vol. 44, No. 6, pp.6–19.

Dingsøyr, T., Moe, N.B. and Seim, E.A. (2018) 'Coordinating knowledge work in multi-team programs: findings from a large-scale agile development program', *Project Management Journal*, Vol. 49, No. 6, pp.64–77.

Einsiedler, S., Mule, S., Rau, C., Hehenberger, P. and Roth, K. (2022) 'A macro-level process model for integrating agile approaches in the design of product-service systems', *International Journal of Agile Systems and Management*, Vol. 15, No. 1, pp.70–92.

Gittell, J.H. (2002) 'Coordinating mechanisms in care provider groups: relational coordination as a mediator and input uncertainty as a moderator of performance effects', *Management Science*, Vol. 48, No. 1, pp.1408–1426.

Gittell, J.H. (2004) 'Coordination networks within and across organizations: a multilevel framework', *Journal of Management Studies*, Vol. 41, No. 1, pp.127–153.

Grimaldi, P., Perrotta, L., Corvello, V. and Verteramo, S. (2016) 'An agile, measurable and scalable approach to deliver software applications in a large enterprise', *International Journal of Agile Systems and Management*, Vol. 9, No. 4, pp.326–339.

Jarzabkowski, P.A., Lê, J.K. and Feldman, M.S. (2012) 'Toward a theory of coordinating: creating coordinating mechanisms in practice', *Organization Science*, Vol. 23, No. 4, pp.907–927.

Kalenda, M., Hyna, P. and Rossi, B. (2018) 'Scaling agile in large organizations: practices, challenges, and success factors', *Journal of Software: Evolution and Process*, Vol. 30, No. 10, e1954.

Kowalczyk, M., Marcinkowski, B. and Przybyłek, A. (2022) 'Scaled agile framework. Dealing with software process-related challenges of a financial group with the action research approach', *Journal of Software: Evolution and Process*, e2455.

Larman, C. and Vodde, B. (2010) *Practices for Scaling Lean and Agile Development: Large, Multisite and Offshore Product Development with Large-Scale Scrum*, Addison Wesley, India.

Leffingwell, D. (2007) *Scaling Software Agility: Best Practices for Large Enterprises*, Addison Wesley, Boston.

Leigh Star, S. (2010) 'This is not a boundary object: reflections on the origin of a concept', *Science, Technology, and Human Values*, Vol. 35, No. 5, pp.601–617.

Licorish, S.A., Holvitie, J., Hyrynsalmi, S., Leppänen, V., Spínola, R.O., Mendes, T.S., MacDonell, S.G. and Buchan, J. (2016) 'Adoption and suitability of software development methods and practices', in Potanin, A., Murphy, G.C., Reeves, S. and Dietrich, J. (Eds.): *2016 23rd Asia-Pacific Software Engineering Conference*, APSEC, IEEE, pp.369–372.

Malone, T.W. and Crowston, K. (1994) 'The interdisciplinary study of coordination', *ACM Computing Surveys*, Vol. 26, No. 1, pp.87–119.

March, J.G. and Simon, H.A. (1958) *Organizations*, John Wiley & Sons, New Jersey.

Okhuysen, G.A. and Bechky, B.A. (2009) '10 coordination in organizations: an integrative perspective', *Academy of Management Annals*, Vol. 3, No. 1, pp.463–502.

Paasivaara, M. and Lassenius, C. (2016) 'Scaling scrum in a large globally distributed organization: a case study', *2016 IEEE 11th International Conference on Global Software Engineering,* ICGSE, Orange County, CA, USA, pp.74–83.

Paasivaara, M., Lassenius, C. and Heikkilä, V.T. (2012) 'Inter-team coordination in large-scale globally distributed scrum: do scrum-of-scrums really work?', *Proceedings of the ACM-IEEE International Symposium on Empirical Software Engineering and Measurement,* ACM-iEEE, pp.235–238.

Razzak, M.A., Richardson, I., Noll, J., Canna, C.N. and Beecham, S. (2018) 'Scaling agile across the global organization: an early stage industrial SAFe self-assessment', *2018 IEEE/ACM 13th International Conference on Global Software Engineering (ICGSE)*, IEEE, Gothenburg, Sweden, pp.116–125.

Rising, L. and Janoff, N.S. (2000) 'The scrum software development process for small teams', *IEEE Software*, Vol. 17, No. 4, pp.26–32.

Šāblis, A., Šmite, D. and Moe, N.B. (2020) 'Team-external coordination in large-scale software development projects', *Evolution and Process*, Advance Online Publication.

Salameh, A. and Bass, J. (2020) 'Heterogeneous tailoring approach using the spotify model', *Proceedings of EASE 2020: Evaluation and Assessment in Software Engineering,* ACM,, Trondheim, Norway pp.293–298.

Scaled Agile Inc. (2021) *Scaled Agile Framework 5*.1, Available at. http://www.scaledagile framework.org (Accessed 1 October, 2021).

Schwaber, K. (2004) *Agile Project Management with Scrum*, Microsoft Press, Berkeley.

Schwaber, K. and Beedle, M. (2001) *Agile Software Development with Scrum*, Prentice-Hall, New York.

Stojanov, I., Turetken, O. and Trienekens, J.J. (2015) 'A maturity model for scaling agile development', *2015 41st Euromicro Conference on Software Engineering and Advanced Applications,* IEEE, Funchal, Portugal, pp.446–453.

Stray, V., Moe, N.B., Strode, D. and Mæhlum, E. (2022) 'Coordination value in agile software development: a multiple case study of coordination mechanisms managing dependencies', *2022 IEEE/ACM 15th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*, IEEE, Pittsburgh, PA, USA, pp.11–20.

Strode, D.E., Huff, S.L., Hope, B. and Link, S. (2012) 'Coordination in co-located agile software development projects', *Journal of Systems and Software*, Vol. 85, No. 6, pp.1222–1238.

Sutherland J. (2001) 'Agile can scale: inventing and reinventing scrum in five companies', *Cutter IT Journal*, Vol. 14, No. 2, pp.5–11.

Taxén, L. and Riedl, R. (2016) 'Understanding coordination in the information systems domain: conceptualization and implications', *Journal of Information Technology Theory and Application*, Vol. 7, No. 1, pp.5–40.

Teixeira, D., Pereira, R., Henriques, T., Mira da Silva, M., Faustino, J. and Silva, M. (2020) 'A maturity model for DevOps', *International Journal of Agile Systems and Management*, Vol. 13, No. 4, pp.464–511.

Thompson, J.D. (1967) *Organizations in Action*, McGraw-Hill, New York.

Uludag, Ö., Philipp, P., Putta, A., Paasivaara, M., Lassenius, C. and Matthes, F. (2022) 'Revealing the state of the art of large-scale agile development research: a systematic mapping study', *Journal of Systems and Software*, pre-print.

Van de Ven, A.H., Delbecq, A.L., Koenig Jr. and R. (1976) 'Determinants of coordination modes within organizations', *American Sociological Review*, Vol. 41, No. 2, pp.322–338.

VersionOne (2021) '15th Annual 'State of agile development', *Survey*, Available at. http://www.stateofagile.com (Accessed 9 October, 2021).

Xu, P. (2009) 'Coordination in large agile projects', *The Review of Business Information Systems*, Vol. 13, No. 4, pp.29–43.

Yin, R.K. (2017) *Case Study Research and Applications: Design and Methods*, 6th ed., SAGE Publications, Thousand Oaks.